

영상처리

3 주차 보고서

컴퓨터융합학부

202102699 정민경

1. 과제 설명

- average_filtering_analysis.py

함수이름	generate_average_filter()
코드설명	Average_filter 를 생성해주는 함수이다. 그렇기 때문에 kernel 을 채워주는데 입력받은 ksize * ksize 만큼의 배열을 1로 채워주고 배열의 크기인 (ksize * ksize) 로 나눠주어 filter 의 합이 1이 되도록 만들어 주었다.
이미지	<pre>def generate_average_filter(ksize): """ 인자 정보 ksize: 커널 크기 return kernel """ ##### # TODO # # average filter 구현 ##### kernel = np.full([ksize, ksize], 1/ksize**2) return kernel</pre>

함수이름	my_filtering()
코드설명	만든 filter 로 직접 filtering 하는 함수이다. 유의미한 시간측정을 위해 filter의 각각의 element 들을 padding한 기존 이미지의 대응하는 element와 곱한 후 이 값을 필터의 중심부분, 즉 기존이미지를 봤을 때 (0, 0) 위치부터 (h, h) 까지 dst 배열에 채워주었다.
이미지	<pre>##### # TODO # # Filtering 구현 # # dst : filtering 결과 image # # 유의미한 시간 측정을 위해 4중 for 문으로 구현 할 것 # # 교수님 이론 PPT 4page 수식 참고 # ##### for row in range(h): for col in range(w): for k_row in range(k_h): for k_col in range(k_w): dst[row, col] += kernel[k_row, k_col] * pad_image[row+k_row, col+k_col]</pre>

- gaussian_analysis1_assignment.py

함수이름	my_filtering()
코드설명	average_filtering_analysis 와는 다르게 유의미한 시간 측정을 할 필요가 없다. 따라서 padding_image에서 filter 크기에 맞는 이미지의 값을 배열 통째로 추출해서 filter와 곱하고, 합계를 낸 후 dst의 (0, 0) 부터 (h,h) 까지 채워준다.
이미지	<pre>def my_filtering(src, mask, pad_type='zero'): (h, w) = src.shape (f_h, f_w) = mask.shape pad_img = my_padding(src, mask, pad_type) dst = np.zeros((h, w)) ##### # TODO 3. Filtering 2중 for문 구현 ##### for row in range(h): for col in range(w): img_filter = pad_img[row: row + f_h, col: col + f_w] dst[row][col] = np.sum(np.multiply(mask, img_filter)) dst = np.round(dst).astype(np.uint8) # 반올림을 해준다. return dst</pre>

함수이름	my_get_gaussian_filter()
코드설명	이 함수는 1D-gaussian filter 를 생성하는 함수이다. 1D 이므로 filter의 행의 개수가 1 일 때와 열의 개수가 1일때를 나누어서 각각의 filter를 return 해준다. 행의 개수가 1일 때는
이미지	<pre>if (f_h == 1): x = np.mgrid[-(f_w // 2): (f_w // 2) + 1, -(f_w // 2): (f_w // 2) + 1][1] gaussian_filter = (1 / (sigma1 * np.sqrt(2 * np.pi))) * np.exp(-(x ** 2) / (2 * sigma1 ** 2)) elif (f_w == 1): y = np.mgrid[-(f_h // 2): (f_h // 2) + 1, -(f_h // 2): (f_h // 2) + 1][0] gaussian_filter = (1 / (sigma1 * np.sqrt(2 * np.pi))) * np.exp(-(y ** 2) / (2 * sigma1 ** 2)) #_mask 총합 1 : 평균 밝기의 변화가 없도록 하기 위함 gaussian_filter = gaussian_filter / np.sum(gaussian_filter) return gaussian_filter</pre>

함수이름	main()
코드설명	Sigma의 값에 따라 gaussian filter 를 시각화 하는 함수이다. 그렇기 때문에 4가지의 sigma의 값에 따라 4가지의 gaussian filter를 생성하고, 이러한 gaussian filter 를 plt.plot 함수를 통해 시각화를 해주었다. Plt.plot 의 첫번째 인자는 x축의 값을, 두번째 인자는 생성한 gaussian filter를, 마지막 인자는 각 그래프의 라벨을 넣어주어, x축과 y축을 설정해 시각화를 해주었다.
이미지	<pre> ##### # TODO 1. sigma 변화에 따른 1D Gaussian filter 시각화 # TODO gaussian_filter 식 채우기 및 그래프 그리기 # TODO 교수님 이론 PPT 40 page 참고 ##### x = np.linspace(-5, 5, 1000)~# -5 ~ 5 사이 동일한 간격의 수 1000개의 1차원 배열 mean = x.mean() sigma1 = 0.5 gaussian_filter1 = (1 / (sigma1 * np.sqrt(2 * np.pi))) * np.exp(-((x - mean) ** 2) / (2 * sigma1 ** 2)) sigma2 = 1 gaussian_filter2 = (1 / (sigma2 * np.sqrt(2 * np.pi))) * np.exp(-((x - mean) ** 2) / (2 * sigma2 ** 2)) sigma3 = 2 gaussian_filter3 = (1 / (sigma3 * np.sqrt(2 * np.pi))) * np.exp(-((x - mean) ** 2) / (2 * sigma3 ** 2)) sigma4 = 3 gaussian_filter4 = (1 / (sigma4 * np.sqrt(2 * np.pi))) * np.exp(-((x - mean) ** 2) / (2 * sigma4 ** 2)) plt.title('Gaussian filter visualization') plt.plot(x, gaussian_filter1, label='sigma=0.5') plt.plot(x, gaussian_filter2, label='sigma=1') plt.plot(x, gaussian_filter3, label='sigma=2') plt.plot(x, gaussian_filter4, label='sigma=3') plt.legend() plt.show() </pre>

- gaussian_analysis2_assignment.py

함수이름	my_padding()
코드설명	Gaussian_analysis1과 동일하게 zero padding으로 이미지를 padding 시켜주는데, 이미지를 기준으로 위, 아래, 왼/오른쪽 총 4가지 방향으로 나누어서 0으로 채워주었다.
이미지	<pre>def my_padding(src, mask, pad_type='zero'): ##### # TODO 4.zero padding 및 repetition padding 구현 ##### # default - zero padding으로 셋팅 (h,w) = src.shape (f_h, f_w) = mask.shape p_h = f_h // 2 p_w = f_w // 2 pad_img = np.zeros((h + p_h * 2, w + p_w * 2)) pad_img[p_h:h + p_h, p_w:w + p_w] = src if pad_type == 'repetition': print('repetition padding') # up pad_img[:p_h, p_w:p_w + w] = src[0, :] # down pad_img[p_h + h:, p_w:p_w + w] = src[h - 1, :] # left pad_img[:, :p_w] = pad_img[:, p_w:p_w + 1] # right pad_img[:, p_w + w:] = pad_img[:, p_w + w - 1: p_w + w] else: # else is zero padding print('zero padding') return pad_img</pre>

함수이름	my_filtering()
코드설명	Gaussian_analysis1과 동일하게 filtering을 시켜주었다. padding_image에서 filter 크기에 맞는 이미지의 값을 배열 통째로 추출해서 filter와 곱하고, 합계를 낸 후 dst의 (0, 0) 부터 (h,h) 까지 채워준다.
이미지	<pre> def my_filtering(src, mask, pad_type='zero'): (h, w) = src.shape (f_h, f_w) = mask.shape pad_img = my_padding(src, mask, pad_type) dst = np.zeros((h, w)) ##### # TODO 3. Filtering 2중 for문 구현 ##### for row in range(h): for col in range(w): img_filter = pad_img[row: row + f_h, col: col + f_w] dst[row][col] = np.sum(np.multiply(mask, img_filter)) dst = np.round(dst).astype(np.uint8) # 반올림을 해준다. return dst </pre>

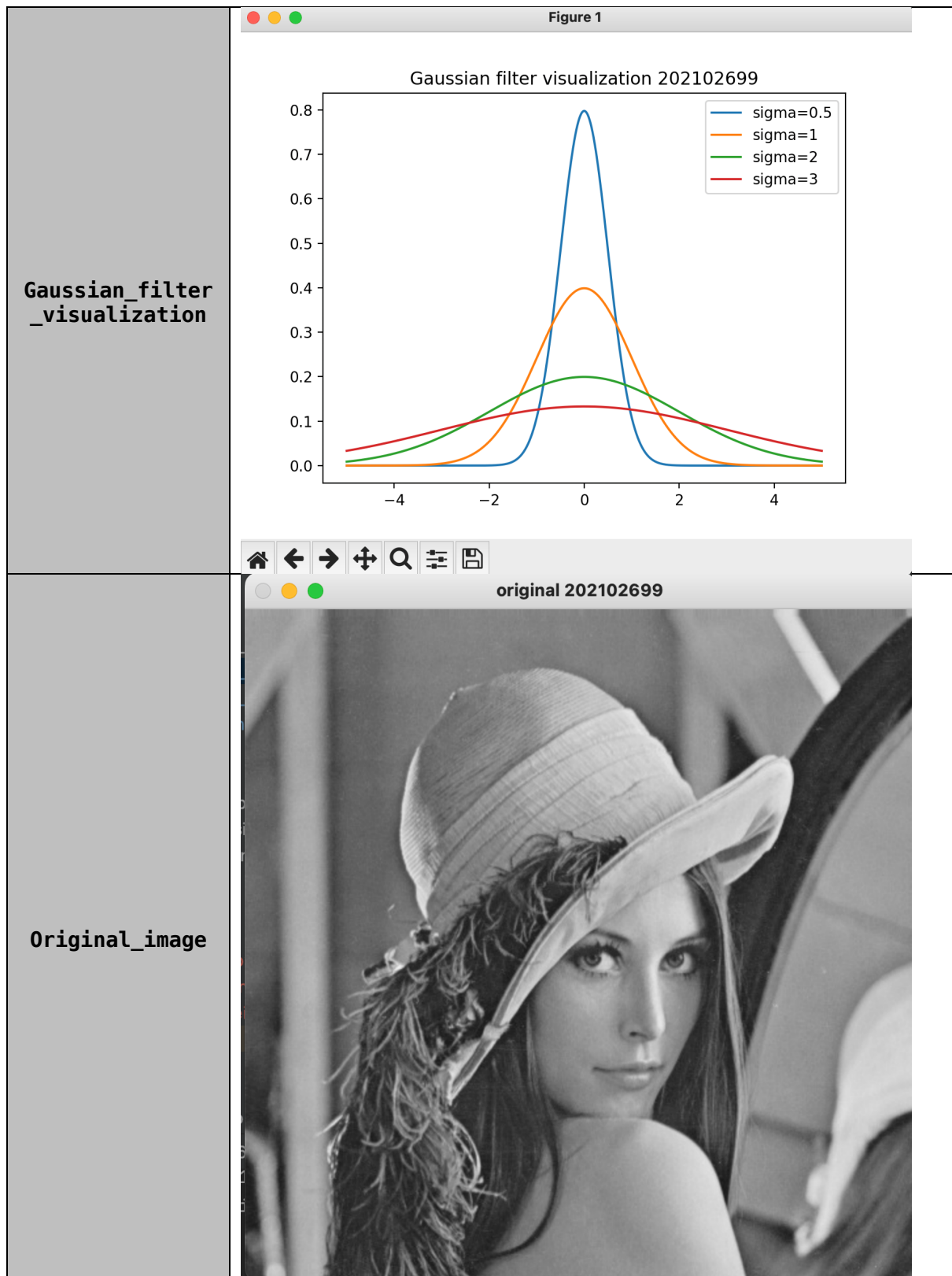
함수이름	my_get_Gaussian_filter()
코드설명	Gaussian_analysis1과 동일하게 gaussian filter를 생성해주었다. 1D와 다른 점은, gaussian filter를 생성해주는 수식에서 x와 y를 나누는 것이 아닌 2D이기 때문에 수식의 exp 안에서 $-(x*x + y*y)$ 로 계산해 2D gaussian filter를 생성해주었다.
이미지	<pre> def my_get_Gaussian_filter(fshape, sigma=1): (f_h, f_w) = fshape ##### # TODO 2 2D Gaussian filter 구현 # TODO 2 np.mgrid를 사용하면 y, x 모두 구할 수 있음 # TODO hint # y, x = np.mgrid[-1:2, -1:2] # y => [[-1, -1, -1], # [0, 0, 0], # [1, 1, 1]] # x => [[-1, 0, 1], # [-1, 0, 1], # [-1, 0, 1]] ##### y, x = np.mgrid[-(f_h // 2): (f_h // 2) + 1, -(f_w // 2): (f_w // 2) + 1] # 2차 gaussian mask 생성 gaussian_filter = np.exp(-((x * x) + (y * y))) / (2 * sigma * sigma) # mask 총합 1 : 평균 밝기의 변화가 없도록 하기 위함 gaussian_filter = gaussian_filter / np.sum(gaussian_filter) return gaussian_filter </pre>

2. 결과 이미지

1. [IP]202102699_average_filtering_analysis

Average_filter_time	<pre>/Users/jeongmingyeong/CNU/IP_23/venv/bin, zero padding 3 X 3 Filter Time : 1.67894675 zero padding 7 X 7 Filter Time : 8.358834958 zero padding 15 X 15 Filter Time : 37.893846208999996 Process finished with exit code 0</pre>
---------------------	--

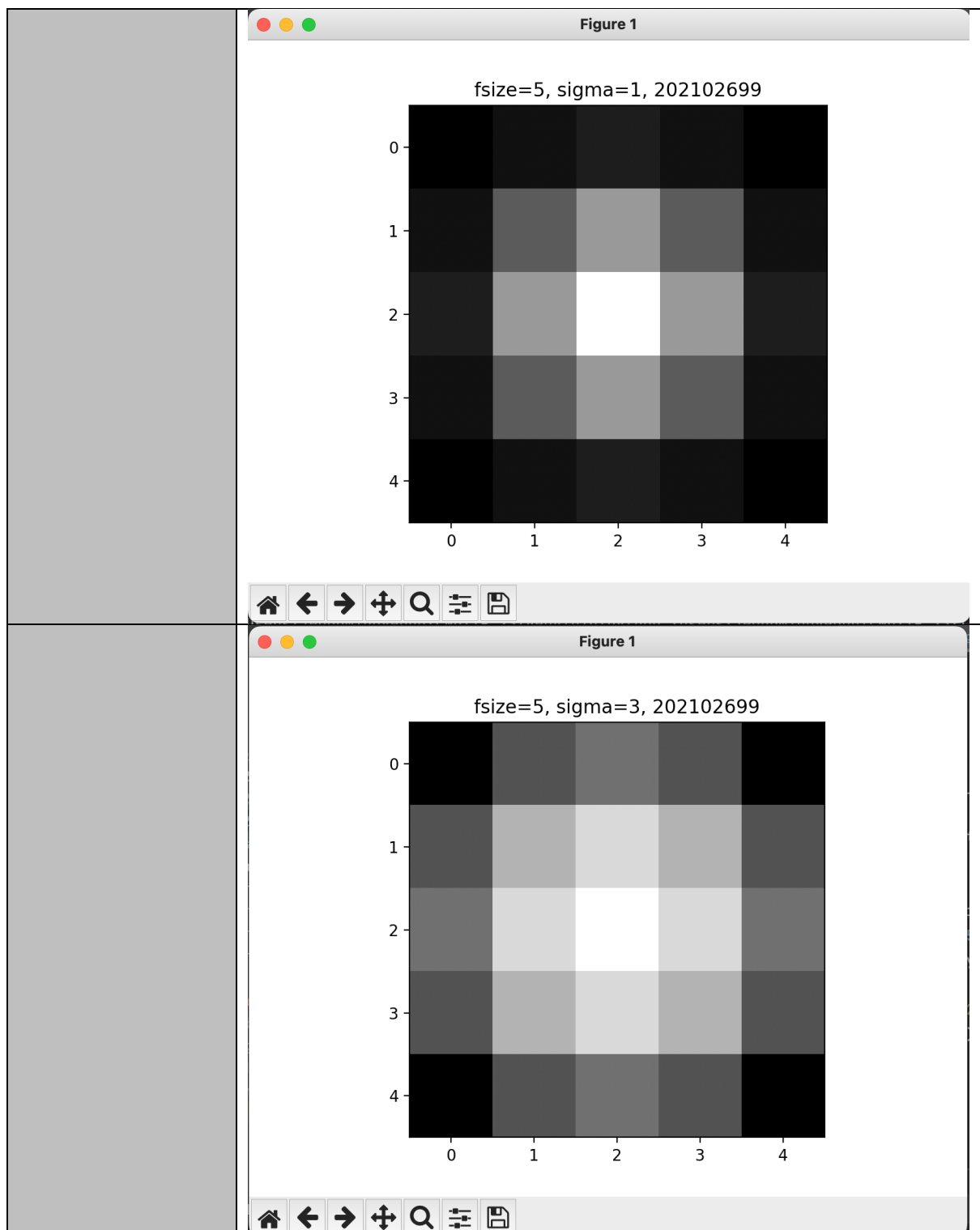
2. [IP]202102699_gaussian_analysis1_assignment

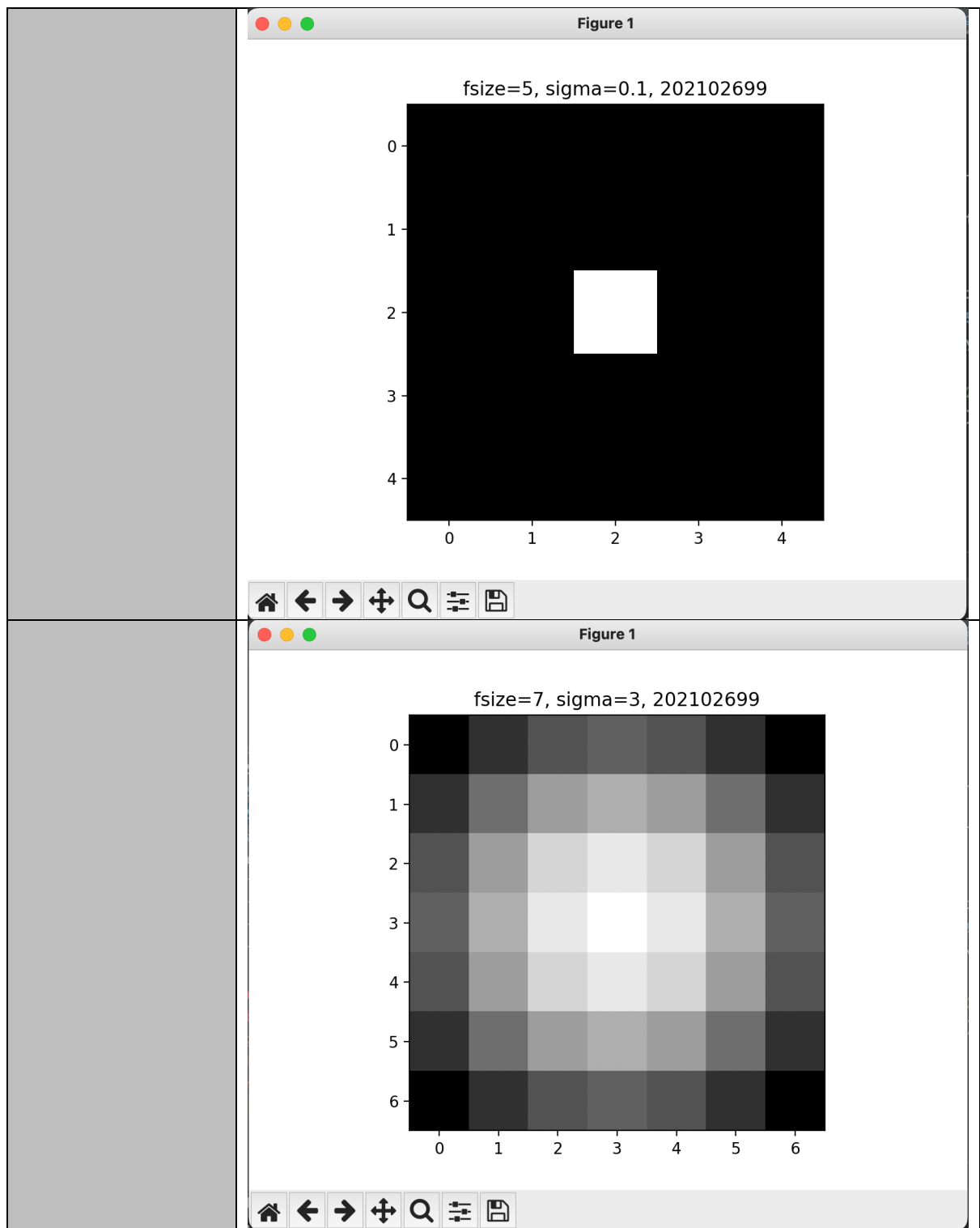


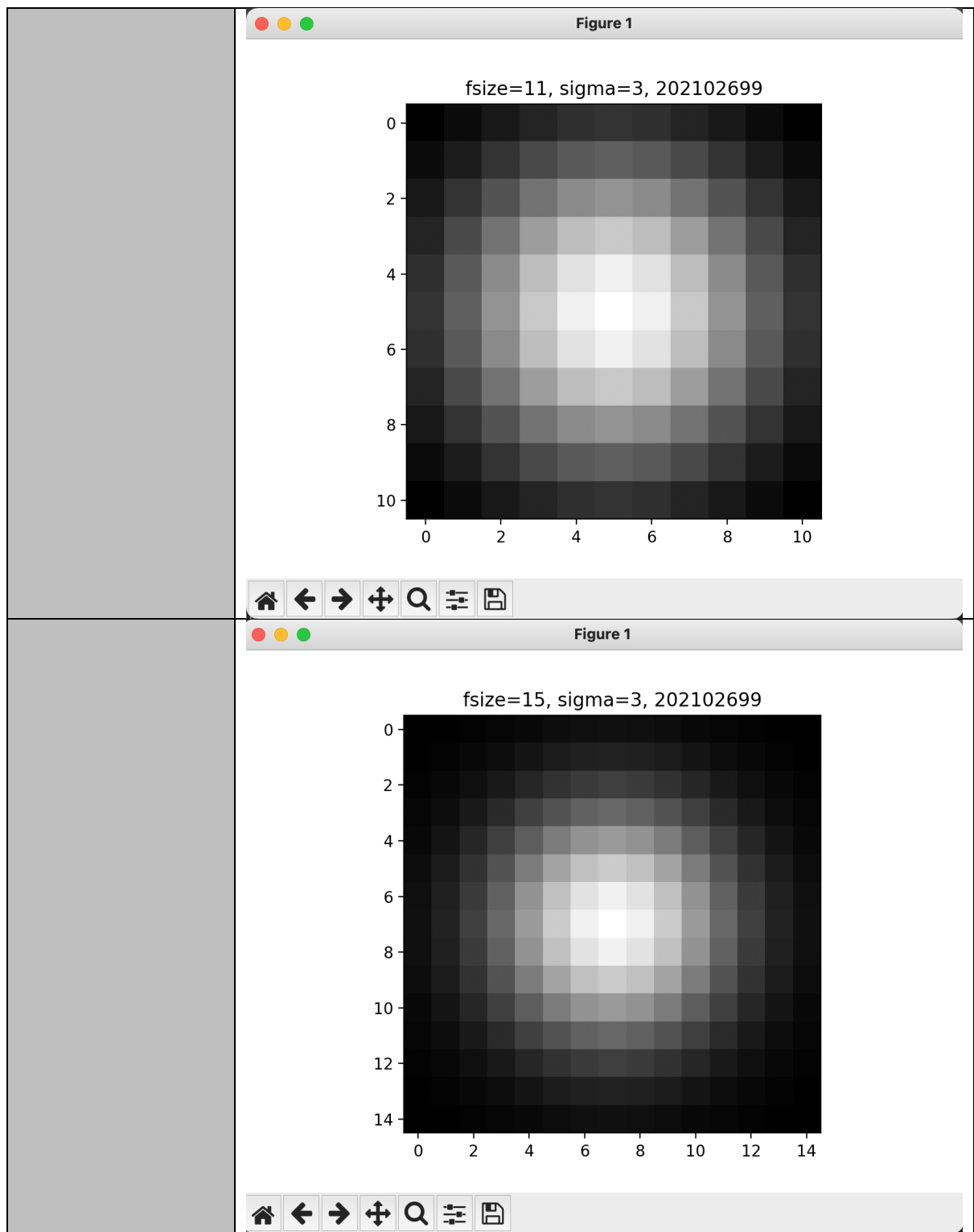
Gaussian_1D
filtering_image



3. [IP]202102699_gaussian_analysis2_assignment







Case 1 Gaussian 2D image 202102699



Case 2 Gaussian 2D image 202102699



Case 3 Gaussian 2D image 202102699



Case 4 Gaussian 2D image 202102699



Case 5 Gaussian 2D image 202102699



Case 6 Gaussian 2D image 202102699



3. 과제 결과 분석

- Average_filter_analysis :

- Filtering 연산에는 아래와 같은 연산이 수행된다.
 - 1) Padding
 - 2) Filtering을 하는 과정에서의 곱셈연산
 - 3) Filtering을 하는 과정에서의 덧셈연산
- Filter의 크기에 따라 연산시간이 차이가 나는 이유는 위에서 언급한 세가지의 연산횟수에서 차이가 나기 때문이다.

[원본 image 의 크기 : $N * M$]

- Filter의 크기가 $3 * 3$ 일 때 :
 - 1) Padding 연산 = 위, 아래, 오른쪽, 왼쪽 모두 1칸씩 패딩으로 채워야 함.
 - 2) Image의 size 만큼의 횟수 * filter의 크기 = $(N * M) * (3 * 3) = 9NM$
 - 3) 각각의 값을 곱셈연산한 값을 더하는 연산 = $9NM + (3 * 3 - 1)NM = 17NM$
- Filter의 크기가 $7 * 7$ 일 때 :
 - 1) Padding 연산 = 위, 아래, 오른쪽, 왼쪽 모두 3칸씩 패딩으로 채워야 함.
 - 2) Image의 size 만큼의 횟수 * filter의 크기 = $(N * M) * (7 * 7) = 49NM$
 - 3) 각각의 값을 곱셈연산한 값을 더하는 연산 = $49NM + (7 * 7 - 1)NM = 97NM$
- Filter의 크기가 $15 * 15$ 일 때 :
 - 1) Padding 연산 = 위, 아래, 오른쪽, 왼쪽 모두 7칸씩 패딩으로 채워야 함.
 - 2) Image의 size 만큼의 횟수 * filter의 크기 = $(N * M) * (15 * 15) = 225NM$
 - 3) 각각의 값을 곱셈연산한 값을 더하는 연산 = $225NM + (15 * 15 - 1)NM = 449NM$
- 이러한 차이 때문에 filter가 커질수록 시간이 점점 더 오래걸리게 된다.

- gaussian_filtering_analysis

Gaussian 1D graph를 보면 sigma 값이 커질수록 그래프가 더 넓게 퍼지는 것을 볼 수 있다. Gaussian 1D graph는 Kernel을 만들 때 가중치를 주는 것을 나타냈는데 sigma값이 작을 수록 image의 중심부분에 더 많은 가중치를 주고, sigma값이 클수록 중심부와 가장자리 부분의 가중치 차이가 작다는 것을 의미한다.

1) 동일 kernel size, sigma 값이 달라질 때 :

kernel의 size가 5로 같은 3개의 filter image에서 sigma만 차이를 둔 3개의 image를 비교해봤을 때 sigma의 값이 작을수록 중심부분과 가장자리 부분의 차이가 큰 것을 확인할 수 있으며, sigma의 값이 클수록 중심부분과 가장자리 부분의 차이가 작아 대체로 밝은 것을 확인할 수 있다.

최종적으로 sigma 가 각각 0.1, 1, 3 인 filter로 filtering한 결과, sigma가 가장 작은 0.1 인 filter는 원본이미지와 가장 유사하게 나온 것을 볼 수있고, sigma가 가장 큰 3인 filter가 가장 많은 smoothing효과가 씌워 사진이 생성된 것을 볼 수 있다.

2) Kernel size 변경, 동일 sigma

Sigma의 값을 3으로 고정시켜놓고 kernel의 size를 7, 11, 15 로 키워가며 비교해봤을 때 전체적인 밝기 상태는 비슷하지만, kernel의 size가 커질수록 kernel 각각이 세부적으로 색이 설정되어 색이 부드럽게 이어지는 것을 볼 수 있다.

결과적으로 kernel의 size가 가장 작은 (7 * 7) filter의 경우보다 (15 * 15) filter로 filtering한 이미지가 더 흐릿하게 나오는 것을 확인할 수 있다. 이는 kernel size가 너무 커서 중심과 멀리 떨어져있는 곳의 값까지 중심에 영향을 주기 때문에 이러한 결과가 나오게 되었다.